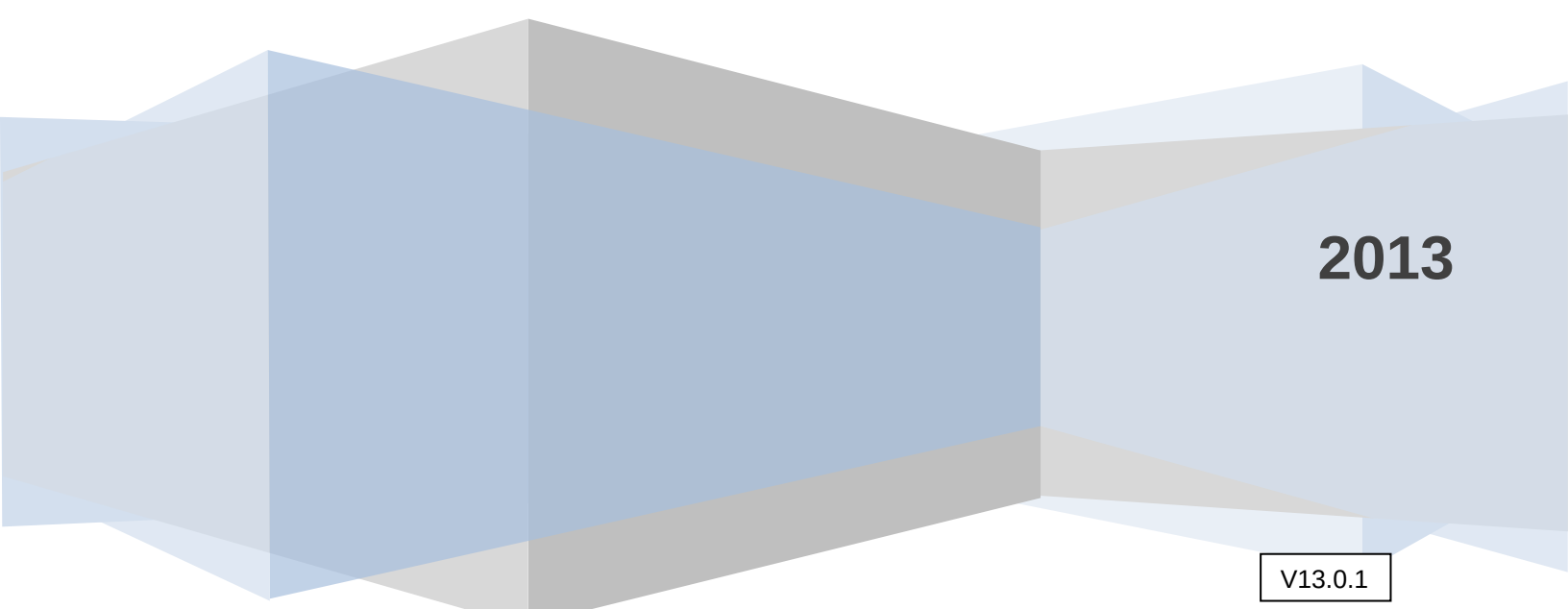


Texas Instruments, Inc.
C2000 Systems and Applications

Digital Motor Control

Software Library:
Special Math Blocks



2013

V13.0.1

Contents

INTRODUCTION 3

eSMO..... 4

PWMwDBC..... 13

Introduction

The digital motor control library is composed of C functions (or macros) developed for C2000 motor control users. These modules are represented as modular blocks in C2000 literature in order to explain system-level block diagrams clearly by means of software modularity. The DMC library modules cover nearly all of the target-independent mathematical macros and target-specific peripheral configuration macros, which are essential for motor control.

In the DMC library, each module is separately documented with source code, use, and background technical theory. The DMC library components have been used by TI to provide system-level motor control examples. In the motor control code, all DMC library modules are initialized according to the system specific parameters, and the modules are inter-connected to each other. At run-time the modules are called in order. Each motor control system is built using an incremental build approach, which allows some sections of the code to be built at a time, so that the developer can verify each section of the application one step at a time. This is critical in real-time control applications, where so many different variables can affect the system, and where many different motor parameters need to be tuned.

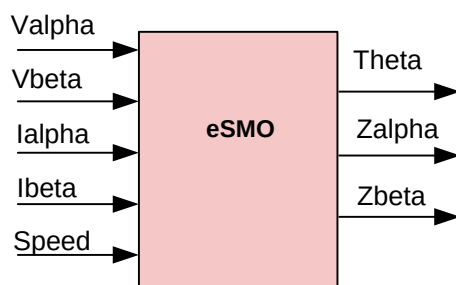
All DMC modules allow users to quickly build, or customize their own systems. The library supports three principal motor types (induction motor, BLDC and PM motors) but is not limited to these motors.

All these modules are provided in the downloadable version of ControlSuite. In addition to these, few additional modules are developed by TI related to the control PMSM motors, for superior performance. The original SMO for sensorless control of PMSM has some scope for improvements and these are covered by enhanced SMO module (eSMO) that provides the following features

1. SMO filter angle compensation
2. Bidirectional speed control
3. Inverter dead band compensation

Description

This software module implements a rotor position estimation algorithm for Permanent-Magnet Synchronous Motor (PMSM) based on Sliding-Mode Observer (SMO) and angle compensation due to sliding filter.

**Availability**

C interface version

Module Properties

Type: Target Independent

Target Devices: 28x Fixed or Floating Point

C Version File Names: smopos.h

IQmath library files for C: IQmathLib.h, IQmath.lib

C Interface

Object Definition

The structure of ESMOPOS object is defined by following structure definition

```
typedef struct {
    _iq Valpha;    // Input: Stationary alpha-axis stator voltage
    _iq Ealpha;    // Variable: Stationary alpha-axis back EMF
    _iq Zalpha;    // Output: Stationary alpha-axis sliding control
    _iq Gsmopos;   // Parameter: Motor dependent control gain
    _iq Estlalpha; // Variable: Estimated stationary alpha-axis stator current
    _iq Fsmopos;   // Parameter: Motor dependent plant matrix
    _iq Vbeta;     // Input: Stationary beta-axis stator voltage
    _iq Ebeta;     // Variable: Stationary beta-axis back EMF
    _iq Zbeta;     // Output: Stationary beta-axis sliding control
    _iq Estlbeta;  // Variable: Estimated stationary beta-axis stator current
    _iq lalpha;    // Input: Stationary alpha-axis stator current
    _iq lalphaError; // Variable: Stationary alpha-axis current error
    _iq Kslide;    // Parameter: Sliding control gain
    _iq lbeta;     // Input: Stationary beta-axis stator current
    _iq lbetaError; // Variable: Stationary beta-axis current error
    _iq Kslf;      // Parameter: Sliding control filter gain
    _iq Theta;     // Output: Compensated rotor angle

    _iq E0;        // Input: Current threshold
    _iq smoFreq;   // Input: SMO filter corner frequency
    _iq smoShift;  // Output: Compensated rotor angle
    _iq runSpeed;  // Input: Running speed of motor
    _iq cmdSpeed;  // Input: Commanded speed of motor
    _iq base_wTs;  // Input: Incremental angle at base frequency
} ESMOPOS;
```

Module Terminal Variables

Item	Name	Description	Format	Range(Hex)
Inputs	Valpha	stationary d-axis stator voltage	GLOBAL_Q	80000000-7FFFFFFF
	Vbeta	stationary q-axis stator voltage	GLOBAL_Q	80000000-7FFFFFFF
	lalpha	stationary d-axis stator current	GLOBAL_Q	80000000-7FFFFFFF
	lbeta	stationary q-axis stator current	GLOBAL_Q	80000000-7FFFFFFF
	E0	Current threshold for smo	GLOBAL_Q	80000000-7FFFFFFF
	runSpeed	Running speed of motor	GLOBAL_Q	80000000-7FFFFFFF
	cmdSpeed	Command speed of motor	GLOBAL_Q	80000000-7FFFFFFF
	base_wTs	Incremental angle at base freq	GLOBAL_Q	80000000-7FFFFFFF
Outputs	Theta	rotor position angle	GLOBAL_Q	00000000-7FFFFFFF (0 – 360 degree)
	Zalfa	stationary d-axis sliding control	GLOBAL_Q	80000000-7FFFFFFF
	Zbeta	stationary q-axis sliding control	GLOBAL_Q	80000000-7FFFFFFF
	smoFreq	Smo filter corner frequency	GLOBAL_Q	80000000-7FFFFFFF
	smoShift	Phase shift due to smo filter	GLOBAL_Q	80000000-7FFFFFFF (0 – 360 degree)
SMOPOS parameter	Fsmopos	$Fsmopos = \exp(-Rs \cdot T/Ls)$	GLOBAL_Q	80000000-7FFFFFFF
	Gsmopos	$Gsmopos = (Vb/lb) \cdot (1 - \exp(-Rs \cdot T/Ls)) / Rs$	GLOBAL_Q	80000000-7FFFFFFF

eSMO			C Interface	
Internal	Kslide	sliding mode control gain	GLOBAL_Q	80000000-7FFFFFFF
	Kslf	sliding control filter gain	GLOBAL_Q	80000000-7FFFFFFF
	Ealpha	stationary d-axis back EMF	GLOBAL_Q	80000000-7FFFFFFF
	Ebeta	stationary q-axis back EMF	GLOBAL_Q	80000000-7FFFFFFF
	Estlalpha	stationary d-axis estimated current	GLOBAL_Q	80000000-7FFFFFFF
	Estlbeta	stationary q-axis estimated current	GLOBAL_Q	80000000-7FFFFFFF
	lalphaError	stationary d-axis current error	GLOBAL_Q	80000000-7FFFFFFF
	lbetaError	stationary q-axis current error	GLOBAL_Q	80000000-7FFFFFFF

* GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

Special Constants and Data types

ESMOPOS

The module definition is created as a data type. This makes it convenient to instance an interface to the sliding-mode rotor position observer of Permanent-Magnet Synchronous Motor module. To create multiple instances of the module simply declare variables of type ESMOPOS.

ESMOPOS_DEFAULTS

Structure symbolic constant to initialize ESMOPOS module. This provides the initial values to the terminal variables as well as method pointers.

Module Usage

Instantiation

The following example instances two ESMOPOS objects
ESMOPOS esmo1, esmo2;

Initialization

To Instance pre-initialized objects
ESMOPOS fe1 = ESMOPOS_DEFAULTS;
ESMOPOS fe2 = ESMOPOS_DEFAULTS;

Invoking the computation macro

```
eSMO_MODULE(&esmo1);
eSMO_MODULE (&esmo2);
```

Example

The following pseudo code provides the information about the module usage.

```
main()
{
    esmo1.Fsmopos = parem1_1;           // Pass parameters to esmo1
    esmo1.Gsmopos = parem1_2;           // Pass parameters to esmo1
    esmo1.Kslide = parem1_3;            // Pass parameters to esmo1
    esmo1.Kslf = parem1_4;              // Pass parameters to esmo1
    esmo2.Fsmopos = parem2_1;           // Pass parameters to esmo2
    esmo2.Gsmopos = parem2_2;           // Pass parameters to esmo2
    esmo2.Kslide = parem2_3;            // Pass parameters to esmo2
    esmo2.Kslf = parem2_4;              // Pass parameters to esmo2
}
```

```
void interrupt periodic_interrupt_isr()
{
    esmo1.Valpha = voltage_dq1.d;           // Pass inputs to esmo1
    esmo1.Vbeta = voltage_dq1.q;           // Pass inputs to esmo1
    esmo1.Ialpha =current_dq1.d;           // Pass inputs to esmo1
    esmo1.Ibeta =current_dq1.q;           // Pass inputs to esmo1
    esmo1.runSpeed =current_speed1;        // Pass inputs to esmo1
    esmo1.cmdSpeed = command_speed1;       // Pass inputs to esmo1

    esmo2.Valpha = voltage_dq2.d;           // Pass inputs to esmo2
    esmo2.Vbeta = voltage_dq2.q;           // Pass inputs to esmo2
    esmo2.Ialpha =current_dq2.d;           // Pass inputs to esmo2
    esmo2.Ibeta =current_dq2.q;           // Pass inputs to esmo2
    esmo2.runSpeed =current_speed2;        // Pass inputs to esmo2
    esmo2.cmdSpeed = command_speed2;       // Pass inputs to esmo2

    eSMO_MODULE(&esmo1)                   // Call compute macro for esmopos1
    eSMO_MODULE(&esmo2);                   // Call compute macro for esmopos2

    angle1 = esmo1.Theta;                  // Access the outputs of esmopos1
    angle2 = esmo2.Theta;                  // Access the outputs of esmopos2
}
```

Technical Background

Figure 1 is an illustration of a permanent-magnet synchronous motor control system based on field orientation principle. The basic concept of field orientation is based on knowing the position of rotor flux and positioning the stator current vector at orthogonal angle to the rotor flux for optimal torque output. The implementation shown in Figure 1 derives the position of rotor flux from encoder feedback. However, the encoder increases system cost and complexity.

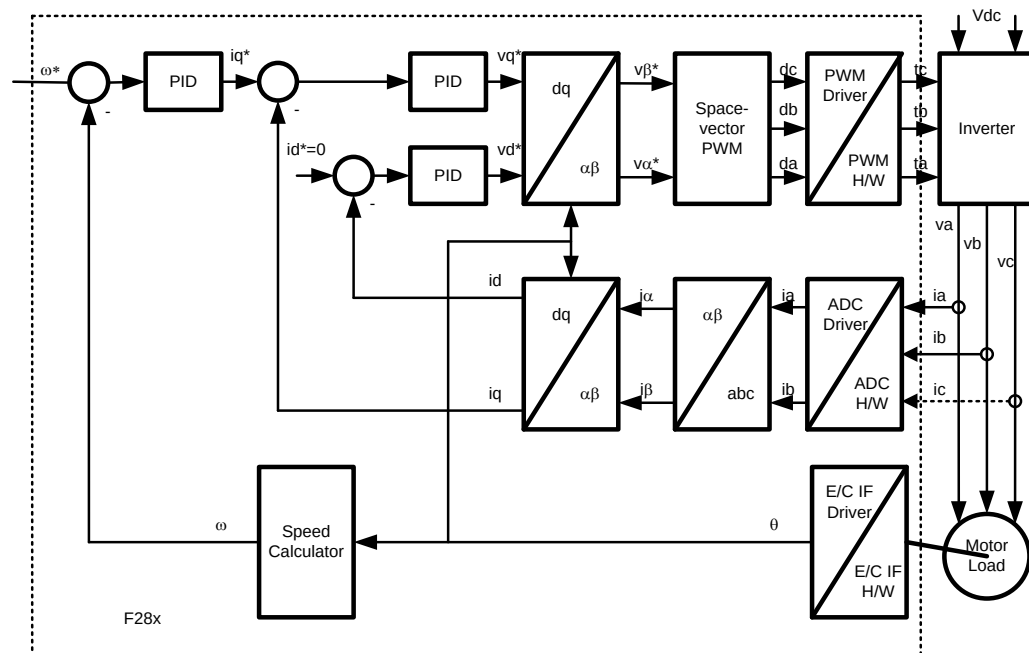


Figure 1 Field Oriented Control of PMSM

Therefore for cost sensitive applications, it is ideal if the rotor flux position information can be derived from measurement of voltages and currents. Figure 2 shows the block diagram of a sensorless PMSM control system where rotor flux position is derived from measurement of motor currents and knowledge of motor voltage commands.

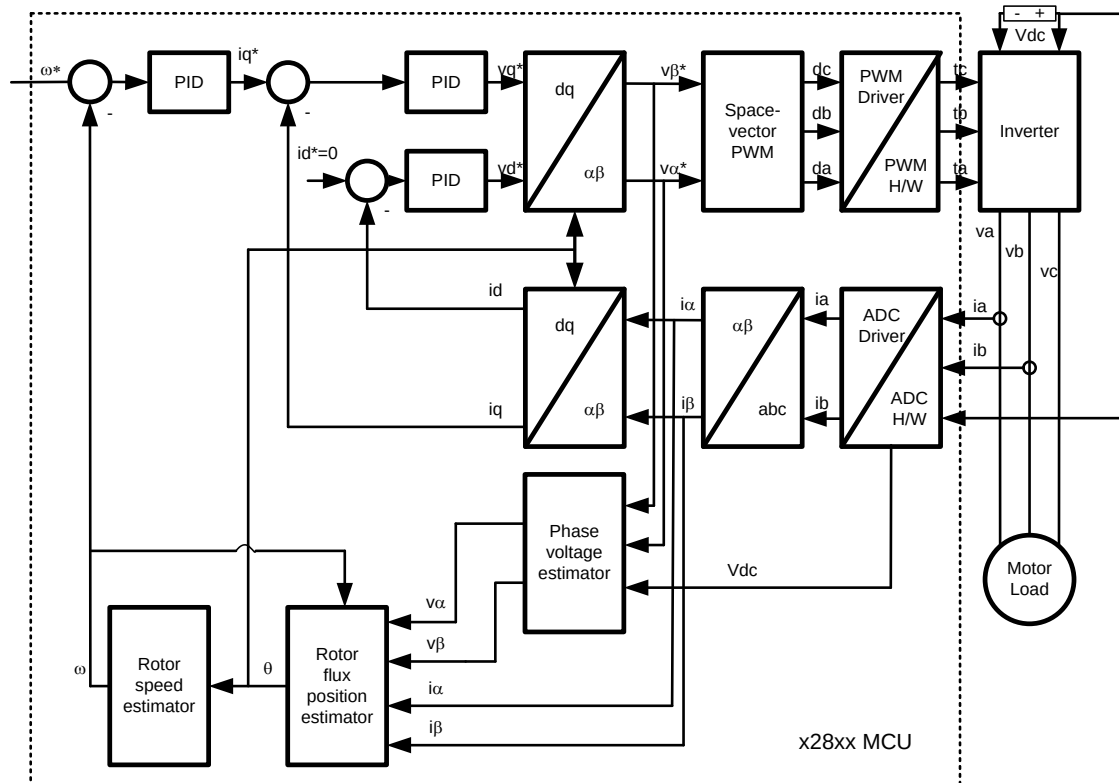


Figure 2 Sensorless Field Oriented Control of PMSM

This software module implements a rotor flux position estimator based on a sliding mode current observer. As shown in Figure 3, the inputs to the estimator are motor phase currents and voltages expressed in α - β coordinate frame.

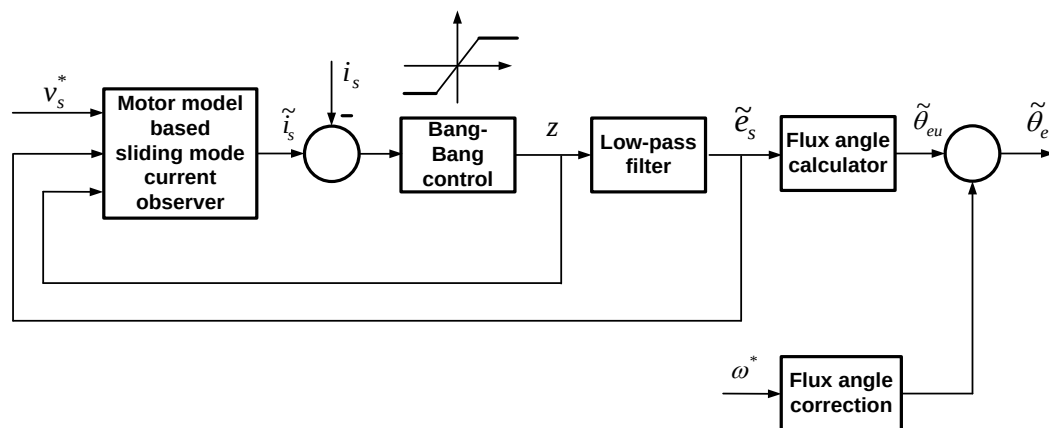


Figure 3 Sliding Mode Observer Based Rotor Flux Position Estimator

Figure 4 is an illustration of the coordinate frames and voltage and current vectors of PMSM, with a , b and c being the phase axes, α and β being a fixed Cartesian coordinate frame aligned with phase a , and d and q being a rotating Cartesian coordinate frame aligned with rotor flux. v_s , i_s and e_s are the motor phase voltage, current and back emf vectors (each with two coordinate entries). All vectors are expressed in α - β coordinate frame for the purpose of this discussion. The α - β frame expressions are obtained by applying Clarke transformation to their corresponding three phase representations.

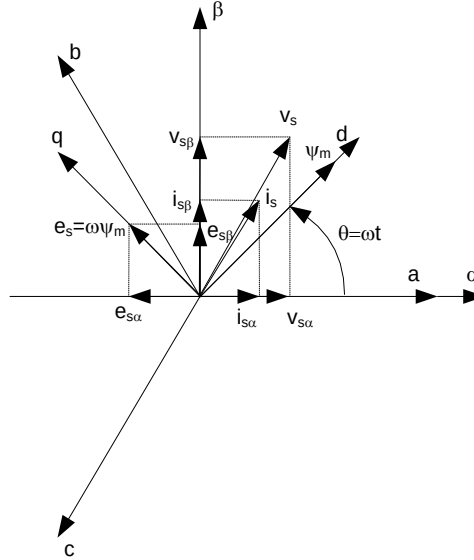


Figure 4 PMSM Coordinate Frames and Vectors

Equation 1 is the mathematical model of PMSM in α - β coordinate frame.

$$\frac{d}{dt} i_s = A i_s + B(v_s - e_s) \quad (1)$$

The matrices A and B are defined as $A = -\frac{R}{L} I_2$ and $B = \frac{1}{L} I_2$ with $L = \frac{3}{2} L_m$, where L_m and R are the magnetizing inductance and resistance of stator phase winding and I_2 is a 2 by 2 identity matrix. Next the mathematical equations for the blocks in Figure 3 are discussed.

1. Sliding Mode Current Observer

The sliding mode current observer consists of a model based current observer and a bang-bang control generator driven by error between estimated motor currents and actual motor currents. The mathematical equations for the observer and control generator are given by Equations 2 and 3.

$$\frac{d}{dt} \tilde{i}_s = A \tilde{i}_s + B(v_s^* - \tilde{e}_s - z) \quad (2)$$

$$z = k \text{sign}(\tilde{i}_s - i_s) \quad (3)$$

The goal of the bang-bang control z is to drive current estimation error to zero. It is achieved by proper selection of k and correct formation of estimated back emf, $\tilde{e}_s$. Note that the symbol $\sim$ indicates that a variable is estimated. The symbol $*$ indicates that a variable is a command.

The discrete form of Equations 2 and 3 are given by Equations 4 and 5.

$$\tilde{i}_s(n+1) = F \tilde{i}_s(n) + G(v_s^*(n) - \tilde{e}_s(n) - z(n)) \quad (4)$$

$$z(n) = k \operatorname{sign}(\tilde{i}_s(n) - i_s(n)) \quad (5)$$

The matrices F and G are given by $F = e^{-\frac{R}{L}T_s} I_2$ and $G = \frac{1}{R}(1 - e^{-\frac{R}{L}T_s})I_2$ where T_s is the sampling period.

2. Estimated Back EMF

Estimated back emf is obtained by filtering the bang-bang control, z , with a first order low-pass filter described by Equation 6.

$$\frac{d}{dt} \tilde{e}_s = -\omega_0 \tilde{e}_s + \omega_0 z \quad (6)$$

The parameter ω_0 is defined as $\omega_0 = 2\pi f_0$, where f_0 represents the cutoff frequency of the filter. The discrete form of Equation 6 is given by Equation 7.

$$\tilde{e}_s(n+1) = \tilde{e}_s(n) + 2\pi f_0 (z(n) - \tilde{e}_s(n)) \quad (7)$$

When the motor speed is lesser than a certain value, the cutoff frequency is kept constant at a certain value. When the motor speed goes above half the cutoff frequency, the cutoff frequency is set to twice the motor speed to ensure reasonable filtering effect.

3. Rotor Flux Position Calculation

Estimated rotor flux angle is obtained based on Equation 8 for back emf.

$$e_s = \frac{3}{2} k_e \omega \begin{pmatrix} -\sin \theta \\ \cos \theta \end{pmatrix} \quad (8)$$

Therefore given the estimated back emf, rotor position can be estimated based on Equation 9.

$$\tilde{\theta}_{eu} = \arctan(-\tilde{e}_{s\alpha}, \tilde{e}_{s\beta}) \quad (9)$$

Since back emf is dependant on the magnitude and polarity of speed, and that arctangent is sensitive to the polarity of its inputs, angle estimate is modified as follows

$$\tilde{\theta}_{eu} = \arctan(-\tilde{e}_{s\alpha} * \operatorname{sign}(\omega), \tilde{e}_{s\beta} * \operatorname{sign}(\omega)) \quad (10)$$

4. SMO Filter Lag Compensation

This is included as part of enhanced SMO. From section 2 above, a first order low pass filter is used to round off the ripples in bemf estimate. Depending on the running speed of motor, it will insert a phase lag as given by

$$\theta_{lag} = \arctan(\omega, \omega_0) \quad (11)$$

By adding this angle with the estimated angle from 3 above, the final estimated angle will closely follow the actual rotor position.

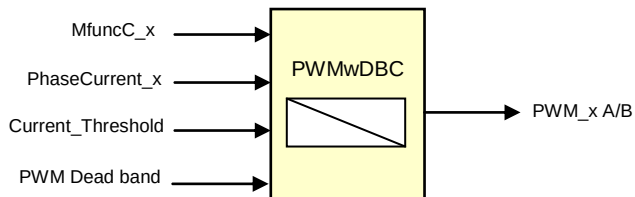
Next, Table 1 shows the correspondence of notations between variables used here and variables used in the program (i.e. smopos.h). The software module requires that both input and output variables are in per unit values.

	Equation Variables	Program Variables
Inputs	$V_{s\alpha}^*$	Valpha
	$V_{s\beta}^*$	Vbeta
	$i_{s\alpha}$	Ialpha
	$i_{s\beta}$	Ibeta
Outputs	$\tilde{\theta}_e$	Theta
	Z_α	Zalpha
	Z_β	Zbeta
Others	$\tilde{i}_{s\alpha}$	EstIalpha
	$\tilde{i}_{s\beta}$	EstIbeta
	$\tilde{e}_{s\alpha}$	Ealpha
	$\tilde{e}_{s\beta}$	Ebeta
	$e^{-\frac{R}{L}T_s}$	Fsmopos
	$\frac{1}{R}(1 - e^{-\frac{R}{L}T_s})$	Gsmopos
	k	Kslide
	$2\pi f_0$	Kslf

Table 1: Correspondence of notations

Description

This module compensates for the effect of dead band in the inverter by adjusting the pulse widths in a feed forward manner based on phase current.


Availability

C interface version

Module Properties

Type: Target Independent

Target Devices: 28x Fixed or Floating Point

C Version File Names: f2803xpwm.h

IQmath library files for C: IQmathLib.h, IQmath.lib

C Interface

This module uses previously defined objects such as PWMGEN and CLARKE and a new object DBC as defined below. Refer to the library for details about PWMGEN and CLARKE objects.

Object Definition

The structure of DBC object is defined by following structure definition

```
typedef struct { _iq Kdtc;           // Input: single consolidated constant
                _iq lth;           // Input: phase current threshold
                _iq scale;         // Input: ratio of pwm dead band to current
                _iq gain;          // Input: switch to add/remove dead band compensation
            } DBC;
```

Item	Name	Description	Format*	Range(Hex)
Inputs	Kdtc	Single consolidated constant	GLOBAL_Q	80000000-7FFFFFFF
	lth	Threshold current for full DBC	GLOBAL_Q	80000000-7FFFFFFF
	Scale	Ratio of PWM DB to lth	GLOBAL_Q	80000000-7FFFFFFF
	Gain	DBC selection switch	GLOBAL_Q	80000000-7FFFFFFF

*GLOBAL_Q valued between 1 and 30 is defined in the IQmathLib.h header file.

Special Constants and Data types

DBC

The module definition is created as a data type. This makes it convenient to instance an interface to dead band compensator. To create multiple instances of the module simply declare variables of type DBC.

DBC_DEFAULTS

Structure symbolic constant to initialize DBC module. This provides the initial values to the terminal variables as well as method pointers.

Module Usage**Instantiation**

The following example instances DBC, PWMGEN and CLARKE objects

```
DBC dbc1;
PWMGEN pwm1;
CLARKE clarke1;
```

To Instance pre-initialized objects

```
DBC dbc1 = DBC_DEFAULTS;
PWMGEN pwm1 = PWM_DEFAULTS;
CLARKE clarke1 = CLARKE_DEFAULTS;
```

Invoking the computation macro

```
PWMwDBC(1, 2, 3, pwm1, clarke1, dbc1);
```

Example

The following pseudo code provides the information about the module usage.

```
main()
{
    Pwm1.PeriodMax = 3000; // PWM freq = 10KHz, clock = 60MHz
    Pwm1.HalfPeriodMax = pwm1.PeriodMax/2;
    PWM_INIT_MACRO(pwm1); // call init macro for pwm1

    dbc1.Ith = thresholdCurrent1; // Pass inputs to dbc1
    dbc1.scale = _IQdiv(Deadband1/2, dbc1.Ith); // Pass inputs to dbc1
    dbc1.gain = _IQ(1.0); // Pass inputs to dbc1
    dbc1.Kdtc = _IQmpy(dbc1.scale, dbc1.gain); //set up correction factor
}

void interrupt periodic_interrupt_isr()
{
    clarke1.As = as1; // phase A current
    clarke1.Bs = bs1; // phase B current
    clarke1.Cs = cs1; // phase C current
    ..
    ..

    pwm1.MfuncC1 = svgen1.Ta;
    pwm1.MfuncC2 = svgen1.Tb;
    pwm1.MfuncC3 = svgen1.Tc;

    PWMwDBC(1,2,3,pwm1,clarke1,dbc1); // Call pwm macro for inverter1
}
```

Technical Background

The significance of dead time in inverters is well known, without which there could be a vertical current shoot through in inverter half bridges leading to excessive power loss in switches and its potential destruction. Dead time represents the time when both high side and low side switches are off. This alters the voltage / pulse width at the inverter output by either reducing or increasing it depending on output current. When the speed is low, the output voltage is low and the pulse width is comparable to dead band. Similarly, at higher PWM frequencies the dead band may represent a significant percentage of PWM period. Under these conditions, the performance is somewhat impaired due to distortions inserted in phase voltages and currents. Hence compensation for the pulse width lost / gained becomes significant to improve the inverter performance.

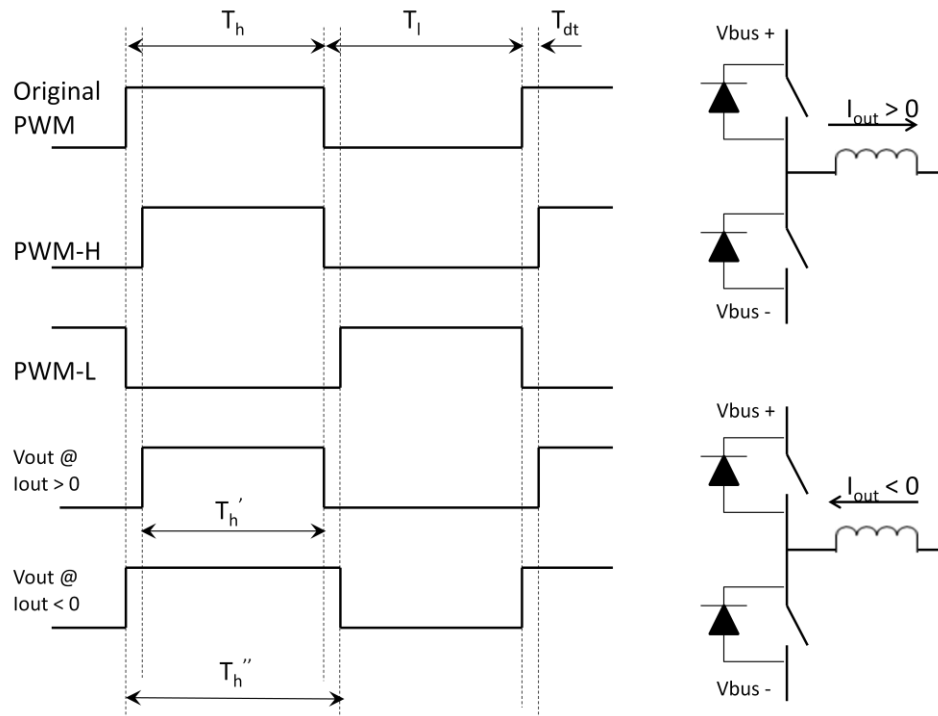


Fig.1 PWM dead band impact on output pulse width

Anytime when the top side switch is ON, output voltage becomes HIGH and when the bottom side switch is ON, output voltage becomes LOW. However, during dead time when both switches are OFF, the output voltage is decided by the polarity of output current that will flow through the appropriate anti parallel diode connected across high and low side switches. If the current direction is positive, i.e., flowing away from the inverter, the output voltage will go LOW by freewheeling through the bottom diode. Likewise, if the output current is flowing into the inverter, the output voltage will go HIGH by freewheeling through the top diode.

From fig.1 above, it is clear that when the current is positive or negative, the output pulse width is reduced or increased by dead time, such that effective output pulse width is $(T_h - T_{dt})$ or $(T_h + T_{dt})$ respectively. Therefore, depending on the output current polarity, a compensation duty cycle is added or subtracted in a feed forward manner that will get cancelled in the inverter. When the output current is small enough for the anti-parallel diodes to fully turn on, adding full compensation can lead to more distortion due to the

fuzzy nature of diode conduction. Hence some prorated compensation is done under these light current conditions as shown in the figure below.

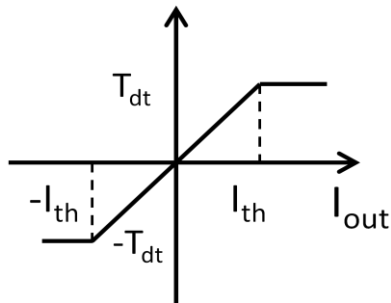


Fig.2 Prorated dead band compensation based on output current

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, modifications, enhancements, improvements, and other changes to its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All products are sold subject to TI's terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by government requirements, testing of all parameters of each product is not necessarily performed.

TI assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using TI components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any TI patent right, copyright, mask work right, or other TI intellectual property right relating to any combination, machine, or process in which TI products or services are used. Information published by TI regarding third-party products or services does not constitute a license from TI to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of TI information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Reproduction of this information with alteration is an unfair and deceptive business practice. TI is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of TI products or services with statements different from or beyond the parameters stated by TI for that product or service voids all express and any implied warranties for the associated TI product or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

TI products are not authorized for use in safety-critical applications (such as life support) where a failure of the TI product would reasonably be expected to cause severe personal injury or death, unless officers of the parties have executed an agreement specifically governing such use. Buyers represent that they have all necessary expertise in the safety and regulatory ramifications of their applications, and acknowledge and agree that they are solely responsible for all legal, regulatory and safety-related requirements concerning their products and any use of TI products in such safety-critical applications, notwithstanding any applications-related information or support that may be provided by TI. Further, Buyers must fully indemnify TI and its representatives against any damages arising out of the use of TI products in such safety-critical applications.

TI products are neither designed nor intended for use in military/aerospace applications or environments unless the TI products are specifically designated by TI as military-grade or "enhanced plastic." Only products designated by TI as military-grade meet military specifications. Buyers acknowledge and agree that any such use of TI products which TI has not designated as military-grade is solely at the Buyer's risk, and that they are solely responsible for compliance with all legal and regulatory requirements in connection with such use.

TI products are neither designed nor intended for use in automotive applications or environments unless the specific TI products are designated by TI as compliant with ISO/TS 16949 requirements. Buyers acknowledge and agree that, if they use any non-designated products in automotive applications, TI will not be responsible for any failure to meet such requirements.